

NATIONAL ECONOMICS UNIVERSITY

Faculty of Information Technology



DATABASE MANAGEMENT SYSTEM

BANKING MANAGEMENT SYSTEM

Submitted by

Full Name	Student ID	Class
Do Pham Ha Chi	11245851	DSEB

Supervised by

PhD. Tran Hung

Hanoi, May 2026

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Project Objectives	4
1.3	Scope and Demonstration Accounts	5
1.4	Out of Scope	6
2	Technology Stack and Selection Rationale	6
2.1	Stack Overview	6
2.2	Why MySQL over a NoSQL store	7
2.3	Why raw SQL instead of an ORM	7
2.4	Why Flask	8
2.5	Why React + Vite and not SSR	8
3	System Analysis and Modelling	8
3.1	Functional Requirements	8
3.2	Non-Functional Requirements	9
4	System Architecture	10
4.1	Three-Tier Overview	10
4.2	Containerisation with Docker Compose	11
5	Project Structure	11
6	Data Generation and Seeding Strategy	15
6.1	Kaggle Dataset	15
6.2	Data Transformation (<code>transform_kaggle_data.py</code>)	16
6.3	Generated Tables	16
6.4	Injected Fraud Signals	16

6.5	Loading Pipeline	17
7	Database Engineering and Schema Design	17
7.1	Entity Relationship Overview	17
8	Detailed Table Specifications	19
8.1	Table Branches	19
8.2	Table Employees	20
8.3	Table Users	20
8.4	Table Customers	21
8.5	Table Accounts	22
8.6	Table Transactions	23
8.7	Table Loans	24
8.8	Table CreditCards and CardTransactions	25
8.9	Table AuditLog	26
9	Transactional Integrity and Automation	27
9.1	Stored Procedure: Atomic Fund Transfer	27
9.2	Trigger: Automated Audit Logging	28
9.3	Indexes and Views	29
10	Backend API	29
10.1	Project Layout	29
10.2	REST Endpoint Catalogue	30
10.3	Application Factory	31
10.4	Authentication Flow	31
10.5	Login Handler	32
10.6	Token Issuing and the <code>auth_required</code> Decorator	32
11	Services Layer	33

11.1 transaction_service	34
11.2 fraud_service	34
11.3 report_service	34
12 Frontend	34
12.1 Project Layout	35
12.2 Routing and Layout	35
12.3 Auth Context and HTTP Client	36
12.4 Forms and Validation	36
12.5 Server-State Management	36
12.6 Visualisations	37
12.7 Visual Snapshots	37
13 Security Posture	42
14 Verification and Testing	42
14.1 Methodology	43
14.2 Testing Strategy	43
14.3 Manual Verification Scenarios	43
15 Conclusion and Future Work	44
15.1 Summary	44
15.2 Future Roadmap	44
Appendices	45
Appendices	45
A Complete Project Directory Structure	45
B Default Demonstration Credentials	45

1 Introduction

1.1 Motivation

Every time a bank teller transfers money between accounts, opens a loan, or flags a suspicious transaction, a chain of coordinated database operations must fire — atomically, securely, and with a full audit trail. Getting any link in that chain wrong means either lost money, a compliance breach, or a silent data corruption that surfaces only when a regulator asks questions.

This project — the **Banking Management System (BMS)** — uses that operational reality as its starting point. Rather than building an abstract toy, the BMS delivers a small but realistic full-stack internal banking platform: a relational database that enforces integrity at the schema level, a Python REST API that applies business rules consistently, a React single-page application that gives staff a clean interface to those operations, and a Python CLI for back-office workflows. Every feature exists because a real bank would need it.

The objective is not to replicate a production core-banking platform, but to demonstrate, end-to-end, how the concepts taught in a Database Management Systems course — normalisation, referential integrity, transactions, stored procedures, triggers, views, indexing, and auditing — combine with a modern web stack to produce a coherent piece of software. Every design decision documented in this report can be traced back to a concrete requirement or a classroom concept, making the system both practically useful and pedagogically transparent.

1.2 Project Objectives

The BMS is built around five interlocking goals that together define its scope.

Data integrity sits at the foundation. The relational schema is designed in Third Normal Form (3NF), eliminating update anomalies and redundancy. Fund movements are implemented as stored procedures that acquire `FOR UPDATE` row locks, ensuring that two concurrent transfers on the same account are serialised rather than interleaved, and that the database never reaches a state where money has left a source account without arriving at a destination.

Auditability is enforced at the database level, not the application level. Every mutating operation on a sensitive table — balance changes, status updates, account closures — fires a trigger that appends an immutable record to the `AuditLog` table, attributed to the authenticated user who initiated the change. Because the trigger lives inside MySQL, no application path can accidentally bypass logging.

Decoupled architecture means that the Flask REST API and the React single-page application share no code and communicate exclusively over HTTP with stateless JWT authentication. Replacing either tier does not require touching the other, and the API can be exercised independently by the Python CLI or by automated tests.

Operational tooling reduces the friction of running the system. A `docker-compose.yml` and a one-click `start.sh` bootstrapper bring up all three containers, run health checks, and seed demonstration data. A Python CLI provides back-office access to MySQL for bulk operations and report generation without opening the web UI.

Analytical value surfaces inside the React application as aggregated reports, a transaction heatmap, a money-flow Sankey diagram, a customer/account network graph, and a rule-based fraud-detection module with an optional scikit-learn classifier trained on historical data.

1.3 Scope and Demonstration Accounts

The BMS is an **internal** system: only bank staff log in, never end customers. Three demonstration accounts are seeded by the installation script so that a reviewer can sign in immediately without creating data manually.

Table 1: Pre-seeded demonstration accounts

Username	Password	Stored Role
admin	admin123	manager
teller1	teller123	teller
audit1	audit123	auditor

Demonstration note. The three accounts exist primarily as login credentials so that reviewers can pick any of them and explore the application. For the purpose of this academic prototype, all three accounts share an identical user experience after sign-in — there is no separate “teller view” or “auditor view” on the home screen. The role value is recorded in the database and embedded in the JWT so that future iterations can introduce role-specific menus and permissions without altering the authentication layer.

1.4 Out of Scope

To keep the project focused, the following items are intentionally excluded: customer self-service portals, mobile applications, real payment-network integrations (SWIFT/ACH), multi-currency FX calculations, regulatory KYC/AML workflows beyond a single ID-number field, and production-grade observability (metrics, distributed tracing).

2 Technology Stack and Selection Rationale

2.1 Stack Overview

Technology choices favoured **maturity, transparency, and a small dependency surface**, so that every layer can be reasoned about during a course defence. The table below provides a comprehensive overview; the subsections that follow explain the reasoning behind the most consequential choices.

Table 2: Detailed Technology Stack

Layer	Technology	Rationale
Database	MySQL 8.0 (InnoDB)	ACID-compliant, row-level locks, JSON columns for audit payloads, native triggers and stored procedures.
DB driver	<code>mysql-connector-python</code>	Raw parameterised SQL through cursors no ORM, full visibility into every query.
Backend API	Python 3.11 + Flask	Minimal micro-framework; blueprints group the eleven resource modules.
Auth	PyJWT (HS256) + Werkzeug <code>pbkdf2:sha256</code>	Stateless tokens with an 8-hour expiry; passwords hashed with PBKDF2-SHA256 plus a per-user salt.
Cross-cutting	Flask-CORS, Flask-Limiter	CORS for the SPA origin; rate limit of 200 req/min per IP.
ML	scikit-learn / joblib	Rule-based and lightweight model-based fraud detection with on-disk persisted artefacts.
Frontend	React 18 + Vite 5	Component-based SPA; Vite gives sub-second HMR during development.
Routing	TanStack React Router	File-style route declarations with type-safe <code><Link></code> ; client-side routing only.

Table 2 continued...

Layer	Technology	Rationale
State	TanStack Query + React Context	Server state with caching and refetching; <code>AuthContext</code> keeps the JWT in memory and <code>localStorage</code> .
HTTP client	axios	Single instance with a request interceptor injecting the <code>Authorization</code> header.
Forms	React Hook Form + Zod	Schema-driven validation with first-class TypeScript inference.
UI / charts	Tailwind CSS, lucide-react, Recharts, D3	Utility-first styling, icon set, standard charts plus advanced visualisations (heatmap, Sankey, network graph).
Toasts	sonner	Non-blocking feedback for create/update/delete actions.
CLI	Python + rich	Terminal-based back-office tool that talks directly to MySQL; used for bulk operations and reports.
Containerisation	Docker + Docker Compose	Three services (mysql, backend, frontend) wired by a single compose file; <code>start.sh</code> drives bring-up, health checks, and seeding.

2.2 Why MySQL over a NoSQL store

A bank's ledger is an inherently relational, strongly-consistent workload. Document stores are convenient for unstructured data but sacrifice the very guarantees needed here: a fund transfer between two accounts must either succeed in full or be rolled back entirely. There is no meaningful concept of “partial consistency” in a double-entry accounting system. MySQL's InnoDB engine provides per-row locking through `SELECT ... FOR UPDATE`, which the `sp_transfer` stored procedure relies on to serialise concurrent transfers touching the same account. The JSON column type in MySQL 8 is used only for audit payloads, where the flexibility of a schema-less blob is genuinely appropriate.

2.3 Why raw SQL instead of an ORM

The original architecture sketch considered SQLAlchemy, but the final implementation uses `mysql-connector-python` with parameterised SQL inside a context-managed cur-

sor. The primary reason is pedagogical transparency: this is a DBMS course project, and an ORM would hide the very queries, joins, and aggregations that the course is designed to teach. Every SQL statement is visible in the route handlers and services, making it straightforward for a reviewer to trace from a business operation (e.g. a fund transfer) to the exact SQL that reaches the database. Parameter binding through the cursor's `%s` placeholders prevents SQL injection without sacrificing readability.

2.4 Why Flask

Flask gives routing, request parsing, error handling, and JSON serialisation — and not much else — which is precisely the appeal. Each blueprint corresponds to one resource (`customers`, `accounts`, `transactions`, and so on), so the project layout mirrors the data model. A heavier framework such as Django would impose conventions and an ORM that conflict with the goal of keeping the data layer explicit.

2.5 Why React + Vite and not SSR

The frontend is a pure single-page application served by Vite in development and as static assets in production. Server-side rendering was considered and rejected for two reasons. First, the application is an internal, authenticated dashboard with no SEO or first-paint requirements that would benefit from SSR. Second, a purely client-side build keeps the deployment trivial: the `frontend` container simply serves static files, and no Node.js process needs to run in production.

3 System Analysis and Modelling

3.1 Functional Requirements

The BMS must satisfy ten categories of functional behaviour, each of which maps directly to one or more database tables, stored procedures, and API endpoints.

1. **Authentication.** Username/password login returns a signed JWT containing `uid`, `uname`, `role`, `iat`, and `exp`. The token is validated on every subsequent request by the `auth_required` decorator.
2. **Customer management.** Full CRUD over the `Customers` table, including the KYC fields required for account opening: full name, national ID number, phone, address, and date of birth.

3. **Account lifecycle.** Staff can open, freeze, and close accounts of three types (*savings*, *checking*, *fixed*), each anchored to a branch. Status transitions are validated server-side and recorded in the audit log.
4. **Core banking operations.** Deposits, withdrawals, and inter-account transfers are implemented as atomic operations. Transfers use the `sp_transfer` stored procedure, which acquires an exclusive row lock before reading the source balance.
5. **Loans and credit cards.** Loan applications generate a monthly repayment schedule at issuance. Credit cards carry a limit, an expiry date, and an independent transaction history in `CardTransactions`.
6. **Branch and employee management.** Branches hold a reference to their managing employee. Employees are linked to a branch and optionally to a `Users` record, allowing staff who have system access to be traced to their physical branch.
7. **Reports and analytics.** Aggregated views expose top customers by balance, branch performance metrics, and daily/weekly transaction volumes. These are consumed directly by the SPA’s chart components.
8. **Audit log.** Every mutating SQL operation is captured by a database-level trigger into the append-only `AuditLog` table. The acting user’s ID is passed via a session variable set by the Flask request hook.

3.2 Non-Functional Requirements

Beyond the functional behaviour, five non-functional properties shape the implementation.

1. **Performance.** Standard CRUD endpoints are expected to respond in well under 200 ms on a single-node MySQL deployment, achievable through the index strategy described in Section 9.3.
2. **Reproducibility.** The full stack is brought up by a single command (`./start.sh`) on any machine with Docker installed. This removes the “works on my machine” problem that plagues multi-developer projects.
3. **Maintainability.** The codebase enforces strict layering: routes call services, services call the database. Business logic is never interleaved with HTTP plumbing, so a change to the transfer rules touches only `transaction_service.py` and the stored procedure, not the route handler.

4. **Auditability.** By placing audit triggers in the database rather than the application, the system guarantees that 100% of writes to ledger tables are captured in `AuditLog`, regardless of which code path initiated the change.
5. **Safety.** All SQL uses bound parameters. Passwords are hashed with PBKDF2-SHA256 and a per-user salt. The JWT secret is read from `.env` and never hardcoded.

4 System Architecture

4.1 Three-Tier Overview

The BMS follows a decoupled three-tier architecture — presentation, business logic, and persistence — with a fourth, optional CLI client that speaks directly to the database for back-office work. This separation is not merely stylistic: it enforces a clear security boundary (MySQL is unreachable from the public network), enables independent scaling of any tier, and makes each layer independently testable.

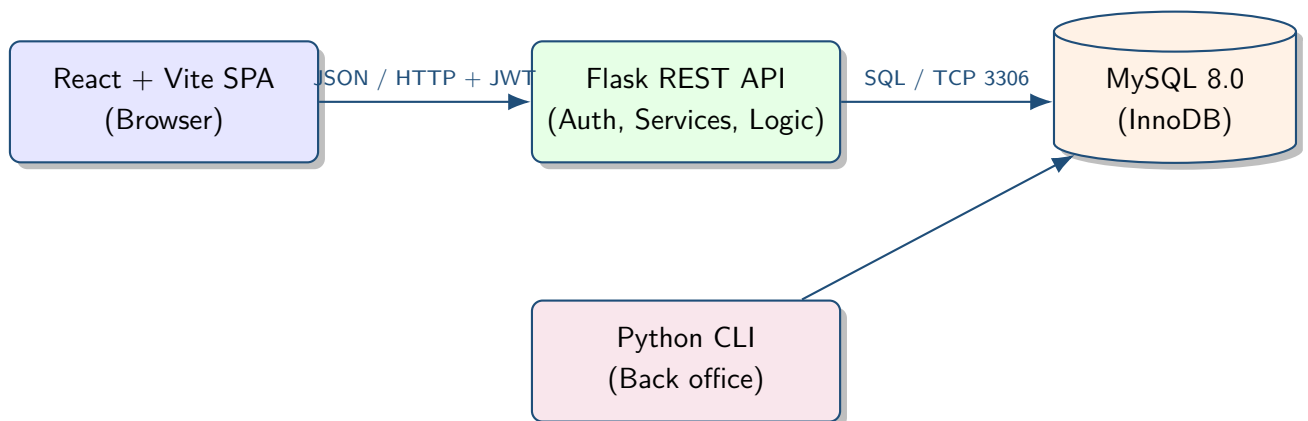


Figure 1: Three-tier architecture with auxiliary administrative CLI

The three-tier split delivers four concrete benefits. First, it enforces separation of concerns: the SPA only renders and forwards user input; the API enforces validation, authentication, and business rules; the database guarantees integrity. Second, it provides security isolation, because MySQL listens only on the internal Docker network and is never reachable from the host’s public interface. Third, the stateless Flask API can be horizontally scaled without changes to the frontend or the database. Fourth, any tier can be replaced independently — for example, substituting Vue for React — without requiring changes to the others.

4.2 Containerisation with Docker Compose

The entire stack is described in `docker-compose.yml` and brought up by `start.sh`, which performs sanity checks, optionally resets the data volume, and waits for each health probe before declaring the system ready. This approach means that a fresh contributor needs only Docker Desktop installed; no local Python, Node.js, or MySQL installation is required.

```

1 $DC up -d
2
3 # Wait for MySQL healthcheck
4 until $DC ps mysql 2>/dev/null | grep -q "healthy"; do
5     sleep 3; (( elapsed += 3 ))
6     (( elapsed >= 120 )) && error "MySQL did not become healthy"
7 done
8
9 # Wait for the Flask API
10 until curl -sf http://localhost:5000/api/health >/dev/null; do
11     sleep 3; (( elapsed += 3 ))
12     (( elapsed >= 90 )) && warn "Backend did not respond in time"
13 done

```

Listing 1: Excerpt of `start.sh` — waiting for MySQL and the API

The three containers each have a distinct responsibility. The `mysql` container runs MySQL 8.0 with the master DDL, triggers, views, and seed scripts mounted into `/docker-entrypoint-initdb.d/`; port 3306 is intentionally not published to the host. The `backend` container runs Flask on port 5000 and reads all secrets from environment variables. The `frontend` container runs Vite on port 5173 and resolves the API base URL from `VITE_API_URL`, making it straightforward to point it at a remote API without any code changes.

5 Project Structure

The repository is organised so that each top-level directory corresponds to one concern: database definitions, backend logic, frontend UI, CLI tooling, and documentation. The layout below reflects the actual file tree, including files that were added or repaired during development (marked **new** or **fixed** in the commentary).

```

1 banking-management-system/
2 |-- core/                                # Shared utilities used by backend +
   CLI
3 |   |-- db_connection.py                 # Connection pool and context-
   managed cursor
4 |   |-- validators.py                    # Input validation helpers
5 |   |-- formatters.py                    # Response formatting utilities

```

```

6 |   '-- auth.py                                # JWT issue / decode /
   auth_required
7 |
8 | -- database/
9 |   |-- schema.sql                            # Table DDL in dependency order
10 |  |-- seed_data.sql                         # Demo accounts and sample data
11 |  |-- views.sql                             # Reporting views
12 |  |-- stored_procedures.sql                 # sp_transfer and other procedures
13 |  |-- functions.sql                         # Scalar helper functions
14 |  |-- triggers.sql                         # Audit triggers on all ledger
   tables
15 |  |-- indexes.sql                           # Performance indexes
16 |  |-- roles_permissions.sql                 # MySQL role definitions
17 |  |-- backup_recovery.sql                   # Backup and recovery scripts
18 |  |-- audit_log.sql                         # AuditLog DDL and trigger helpers
19 |  '-- load_csv.sql                          # LOAD DATA INFILE from csv_output/
20 |
21 | -- backend/
22 |   |-- app.py                               # create_app() factory
23 |   |-- config.py                           # Environment-driven settings
24 |   |-- api_response.py                      # ok() / fail() / paginate() helpers
25 |   |-- requirements.txt                     # Python dependencies
26 |   |-- routes/
27 |     |-- auth_routes.py                     # /login, /logout, /me
28 |     |-- customer_routes.py                 # CRUD for customers
29 |     |-- account_routes.py                  # Account open/freeze/close
30 |     |-- transaction_routes.py              # deposit / withdraw / transfer
31 |     |-- employee_routes.py                 # Staff management
32 |     |-- branch_routes.py                   # Branch CRUD
33 |     |-- loan_routes.py                     # Loan applications and schedules
34 |     |-- card_routes.py                     # Credit cards and card transactions
35 |     |-- report_routes.py                   # Analytics endpoints
36 |     |-- audit_routes.py                    # Audit log viewer
37 |   |-- services/
38 |     |-- auth_service.py                    # Login logic and token helpers
39 |     |-- transaction_service.py              # Deposit/withdraw/transfer wrappers
40 |     |-- fraud_service.py                   # Rule-based fraud detection
41 |     |-- report_service.py                  # Aggregation queries
42 |   |
43 |   |-- ml/
44 |     |-- fraud_detection_model.py           # Detector singleton loaded at
startup
45 |   |-- utils/
46 |     |-- query_builder.py                   # Whitelisted dynamic filter/sort
builder
47 |   '-- tests/
48 |     |-- test_auth.py                       # Auth unit tests
49 |     |-- test_customers.py                  # Customer CRUD tests
50 |     '-- test_transactions.py               # Concurrency and balance tests
51 |
52 | -- frontend/

```

```

53 |   '-- src/
54 |       |-- main.tsx
55 |       |-- routeTree.tsx
56 |       |-- index.css           # Tailwind directives and theme
57 |   tokens
58 |       |-- contexts/
59 |       |   '-- AuthContext.tsx # JWT in memory + localStorage
60 |       |-- services/
61 |       |   |-- api.ts          # Axios instance with interceptors
62 |       |   |-- authService.ts  # login / logout / me calls
63 |       |   |-- customerService.ts # Customer API wrapper
64 |       |   |-- accountService.ts # Account API wrapper
65 |       |   '-- transactionService.ts # Transaction API wrapper
66 |       |-- components/
67 |       |   |-- AppLayout.tsx   # Sidebar, topbar, and Outlet
68 |       |   |-- Protected.tsx   # Auth + role guard
69 |       |   |-- Modal.tsx
70 |       |   |-- StatCard.tsx
71 |       |   |-- Pagination.tsx
72 |       |   |-- FilterBar.tsx
73 |       |   |-- Skeleton.tsx
74 |       |   |-- LoginForm.tsx
75 |       |   |-- TransactionForm.tsx
76 |       |   '-- charts/
77 |       |       |-- MoneyFlowSankey.tsx # D3 Sankey diagram
78 |       |       |-- NetworkGraph.tsx    # D3 force-directed graph
79 |       |       '-- TransactionHeatmap.tsx # Calendar heatmap
80 |       |-- hooks/
81 |       |   |-- use-mobile.tsx
82 |       |   |-- useDebounce.ts
83 |       |   |-- useFilters.ts
84 |       |   |-- useAuth.ts          # Auth context consumer
85 |       |   '-- useToast.ts         # Wrapper around sonner
86 |       |-- routes/
87 |       |   |-- login.tsx
88 |       |   |-- dashboard.tsx
89 |       |   |-- customers.tsx
90 |       |   |-- accounts.tsx
91 |       |   |-- transactions.tsx
92 |       |   |-- employees.tsx
93 |       |   |-- branches.tsx
94 |       |   |-- loans.tsx
95 |       |   |-- cards.tsx
96 |       |   |-- audit.tsx
97 |       |   |-- reports.tsx
98 |       |   '-- reports.advanced.tsx
99 |   '-- lib/
100 |       |-- utils.ts             # cn() and shared helpers
101 |       |-- validations.ts       # Zod schemas for all forms
102 |       '-- mock-data.ts        # Legacy stubs

```

```

103 |
104 |-- data_generation/
105 |   |-- download_kaggle.py           # Kaggle dataset downloader
106 |   |-- transform_kaggle_data.py     # Data transformation
107 |   |-- generate_employees.py        # Synthetic employee generator
108 |   |-- inspect_schema.py           # Schema introspection helper
109 |   '-- csv_output/                  # Generated CSVs loaded by load_csv.
      sql
110 |       |-- customers.csv
111 |       |-- accounts.csv
112 |       |-- transactions.csv
113 |       |-- branches.csv
114 |       '-- employees.csv
115 |
116 |-- python_cli/
117 |   |-- main.py                     # Rich-powered interactive menu
118 |   |-- auth_cli.py                 # CLI login and session management
119 |   |-- operations/                 # Deposit, withdraw, transfer, CRUD
      ops
120 |   '-- reports/                     # CLI report generators
121 |
122 |-- docs/
123 |   |-- project_brief.pdf
124 |   |-- api_documentation.md
125 |   |-- er_diagram.png             # Exported from MySQL Workbench
126 |   |-- architecture_diagram.png
127 |   |-- user_manual.md
128 |   '-- screenshots/                # UI evidence for the report
129 |       |-- dashboard.png
130 |       |-- customers.png
131 |       |-- transactions.png
132 |       '-- reports.png
133 |
134 |-- tests/                          # End-to-end tests
135 |   |-- test_login.spec.ts
136 |   |-- test_customer_crud.spec.ts
137 |   '-- test_transaction_flow.spec.ts
138 |
139 |-- .env.example                     # Template with all required keys
140 |-- .env.development                 # Development overrides
141 |-- .env.production                  # Production overrides
142 |-- .gitignore
143 |-- docker-compose.yml              # Three-service composition
144 |-- Dockerfile.backend              # Python 3.11 image for Flask API
145 |-- Dockerfile.frontend             # Node 20 image for Vite
146 |-- docker-entrypoint.sh            # Container startup script
147 |-- README.md                       # Setup guide and feature overview
148 |-- run_windows.bat                  # Windows launcher
149 '-- run_linux.sh                    # Linux/macOS launcher

```

Listing 2: Complete project directory tree

Key design decisions in the layout. The `core/` package sits outside `backend/` so that both the Flask API and the Python CLI can import the same database connection code and validators without duplication. The `database/` scripts are numbered and named so they can be executed in order, with `schema.sql` always running before `triggers.sql` (which references the tables that schema defines). The `data_generation/` directory separates the one-time data preparation pipeline from the application code, keeping the CSV output under version control so the seeded database state is reproducible.

6 Data Generation and Seeding Strategy

A realistic demonstration database is critical for validating the system’s reporting, fraud detection, and analytics features. Rather than hand-crafting synthetic data, the BMS seeds its database from a combination of a publicly available Kaggle dataset and purpose-built generation scripts that fill in the gaps.

6.1 Kaggle Dataset

The primary data source is the *Synthetic Banking Dataset* published on Kaggle ([akrambelha/synthetic](#)) which provides a large, realistic starting point.

Table 3: Kaggle source tables and record counts

Table	Records	Status
customers	50,000	Used directly (transformed)
accounts	75,000	Used directly (transformed)
cards	100,000	Used directly (transformed)
merchants	5,000	Reference data
branches	500	Used directly (transformed)
loans	30,000	Used directly (transformed)
transactions	1,000,000	Partially used; augmented
Total	1,260,500	

6.2 Data Transformation (`transform_kaggle_data.py`)

Because the Kaggle schema does not map directly to the BMS schema, a transformation pipeline in `data_generation/transform_kaggle_data.py` reads the raw CSVs and produces schema-compatible outputs in `csv_output/`. The pipeline handles column renaming, type coercion, deduplication of emails and ID numbers, and range validation (e.g. ensuring all balances are positive). Environment variables (`N_BRANCHES`, `N_TX_PER_ACC`, `FRAUD_RATE`) allow the dataset size to be tuned without touching the code.

6.3 Generated Tables

Two tables have no equivalent in the Kaggle source and are generated entirely from scripts.

Employees (`transform_kaggle_data.py`). Approximately 5,000 employee rows are generated with the Faker library, distributed across the 500 branches at roughly 10 employees per branch. Positions (Manager, Auditor, Loan Officer, Teller, Cashier) are sampled with realistic weights, and salaries are drawn from position-specific ranges. This ensures the employee–branch relationship is populated for reporting queries.

Card Transactions (`generate_card_transactions.py`). For every active or recently expired card, between 8 and 15 card-transaction rows are generated with randomised amounts, merchant names, and statuses (approved / declined / reversed). The script keys transactions to the sequential `CardID` produced by MySQL's `AUTO_INCREMENT` after the cards CSV is loaded, so referential integrity is guaranteed.

6.4 Injected Fraud Signals

To give the fraud-detection module meaningful training data, the transaction generator deliberately injects four types of suspicious patterns into roughly 1.5% of accounts: *burst withdrawals* (6–12 rapid ATM withdrawals minutes apart), *amount spikes* (single withdrawal of \$15,000–\$49,000), *late-night activity* (3–6 withdrawals between midnight and 04:00), and *structuring* (repeated deposits just under \$10,000 to simulate the classic structuring pattern). These signals are labelled implicitly by the rule-based fraud detector and explicitly by the ML training pipeline.

6.5 Loading Pipeline

After transformation, `database/load_csv.sql` bulk-loads all CSV files with `LOAD DATA LOCAL INFILE`, inserting roughly 1.26 million rows in the order dictated by foreign-key dependencies (branches → employees → customers → accounts → transactions → loans → cards → card transactions). The ML training script `backend/ml/train_models.py` then reads the loaded transactions directly from MySQL to fit and persist the fraud classifier.

7 Database Engineering and Schema Design

7.1 Entity Relationship Overview

The schema centres on five core business entities — branches, employees, customers, accounts, and transactions — supplemented by loans, credit cards (with their own card transaction history), the system users table, and a global audit log. Figure 2 shows the high-level relationships; the detailed per-table specifications follow in Section 8.

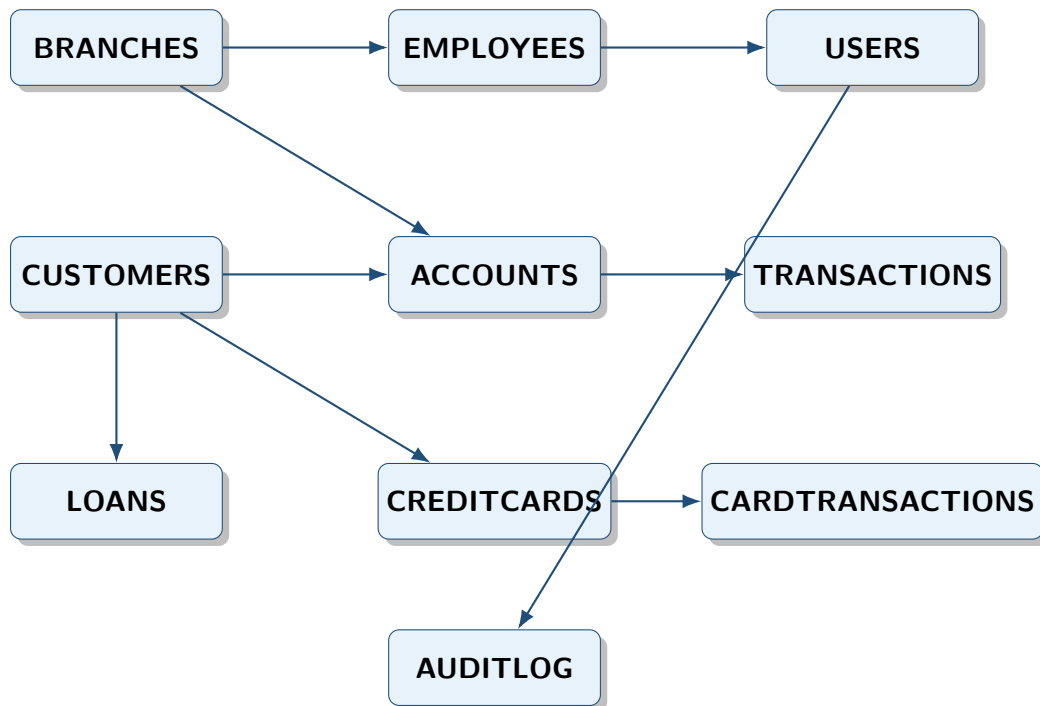


Figure 2: High-level entity topology

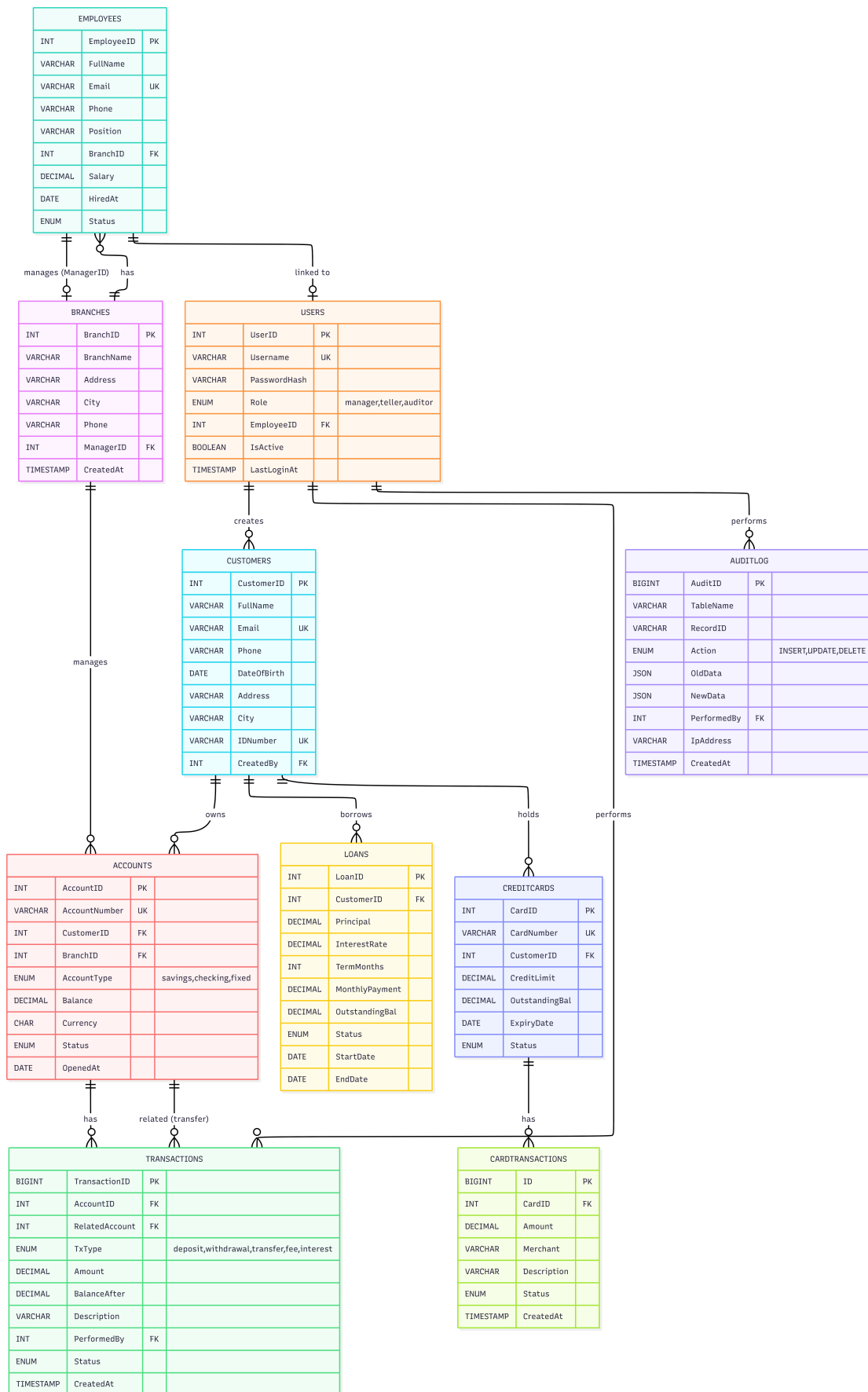


Figure 3: Full crow's-foot ERD exported from MySQL Workbench.

Primary keys (PK) and foreign keys are connected by relationship lines with cardinality annotations. **Accounts** is the central hub, linking upward to **Branches** and **Customers** and downward to **Transactions**, **Loans**, and **CreditCards**. **AuditLog** stands apart with no outgoing foreign keys, reinforcing its append-only, tamper-evident role.

8 Detailed Table Specifications

This section describes every column of every table, the constraints applied, and the business reasoning behind each design choice. Primary keys use auto-incrementing surrogate integers throughout to decouple the physical row identifier from the business identifier (account number, national ID) and to simplify foreign key references.

8.1 Table Branches

A branch represents a physical office in the bank’s network. The **ManagerID** column is a self-referential foreign key into **Employees**, nullable because a branch may temporarily have no assigned manager. The `ON DELETE SET NULL` rule prevents orphaned records when a manager’s employee row is deleted.

Table 4: Branches

Column	Type	Notes
BranchID	INT PK AUTO_INCREMENT	Surrogate key.
BranchName	VARCHAR(100) NOT NULL	Display name shown in the UI and reports.
Address	VARCHAR(255) NOT NULL	Street address for customer-facing correspondence.
City	VARCHAR(80) NOT NULL	Enables geographic grouping in branch performance reports.
Phone	VARCHAR(20)	Optional contact number.
ManagerID	INT FK → Employees	Nullable; ON DELETE SET NULL to handle departures.
CreatedAt	TIMESTAMP DEFAULT NOW()	Immutable audit timestamp.

8.2 Table Employees

Employees are the human actors within the bank. Each employee belongs to exactly one branch (**BranchID** NOT NULL), and the **Status** ENUM encodes three lifecycle states: *active* (can log in), *suspended* (temporarily prevented from acting), and *terminated* (historical record retained for audit). Storing salary as **DECIMAL(12,2)** avoids the binary floating-point rounding errors that would occur with **FLOAT**.

Table 5: Employees

Column	Type	Notes
EmployeeID	INT PK AUTO_INCREMENT	Surrogate key.
FullName	VARCHAR(120) NOT NULL	Legal full name.
Email	VARCHAR(120) UNIQUE NOT NULL	Corporate email; enforced unique by index.
Phone	VARCHAR(20)	Optional direct line.
Position	VARCHAR(60) NOT NULL	Job title for display and role audits.
BranchID	INT FK → Branches NOT NULL	Home branch; cannot be null.
Salary	DECIMAL(12,2) NOT NULL CHECK ≥ 0	Stored as DECIMAL to avoid floating-point rounding.
HiredAt	DATE NOT NULL	Hire date; used in tenure calculations.
Status	ENUM NOT NULL DEFAULT 'active'	<i>active, suspended, terminated.</i>

8.3 Table Users

Users holds login credentials and links each system account optionally to an employee record via a **UNIQUE** foreign key, enforcing the one-user-per-employee constraint at the database level. The **IsActive** flag enables soft disabling — the employee record is preserved for historical queries, but the user can no longer authenticate. **LastLoginAt**

is updated by the login route and provides an operationally useful audit trail of access patterns.

Table 6: Users

Column	Type	Notes
<code>UserID</code>	INT PK AUTO_INCREMENT	Surrogate key.
<code>Username</code>	VARCHAR(60) UNIQUE NOT NULL	Login handle.
<code>PasswordHash</code>	VARCHAR(255) NOT NULL	Werkzeug <code>pbkdf2:sha256</code> hash with per-user salt.
<code>Role</code>	ENUM NOT NULL	<i>manager, teller, auditor</i> — embedded in JWT.
<code>EmployeeID</code>	INT FK → Em- ployees UNIQUE	UNIQUE enforces one user per employee.
<code>IsActive</code>	BOOLEAN NOT NULL DEFAULT TRUE	Soft disable without deleting the row.
<code>LastLoginAt</code>	TIMESTAMP	Updated on each successful login.

Warning: The `PasswordHash` column is wide enough for the full PBKDF2 output (algorithm, iteration count, salt, and digest). Hashes are never returned by the API and never written to the audit log.

8.4 Table Customers

The `Customers` table holds the KYC data required to open an account. `IDNumber` carries a `UNIQUE` constraint because a national ID should map to at most one customer record; a duplicate would indicate either a data entry error or an attempted identity fraud. `CreatedBy` records which system user onboarded the customer, providing traceability back to an individual employee for compliance purposes.

Table 7: Customers

Column	Type	Notes
CustomerID	INT PK AUTO_INCREMENT	Surrogate key.
FullName	VARCHAR(120) NOT NULL	Legal name.
Email	VARCHAR(120)	Optional; used for notifications.
Phone	VARCHAR(20) NOT NULL	Primary contact number.
DateOfBirth	DATE NOT NULL	Used for minimum-age checks at account opening.
Address	VARCHAR(255) NOT NULL	Residential address.
City	VARCHAR(80) NOT NULL	Enables geographic customer reports.
IDNumber	VARCHAR(30) UNIQUE NOT NULL	National ID; mandatory KYC field.
CreatedBy	INT FK → Users NOT NULL	The user who onboarded this customer.

8.5 Table Accounts

An account is the central entity in the banking domain. The **Balance** column is maintained exclusively by the stored procedures and services layer, never by direct **UPDATE** statements in route handlers, to ensure that every balance change passes through the validation and locking logic. **AccountNumber** is a human-readable, unique public identifier generated at account opening; it is separate from the surrogate **AccountID** used in foreign keys.

Table 8: Accounts

Column	Type	Notes
AccountID	INT PK AUTO_INCREMENT	Surrogate key used in all foreign references.
AccountNumber	VARCHAR(20) UNIQUE NOT NULL	Human-readable public identifier.
CustomerID	INT FK → Customers NOT NULL	Account owner.
BranchID	INT FK → Branches NOT NULL	Hosting branch for geographic reporting.
AccountType	ENUM NOT NULL	<i>savings, checking, fixed.</i>
Balance	DECIMAL(18,2) NOT NULL DEFAULT 0	Always non-negative; enforced by procedure logic.
Currency	CHAR(3) NOT NULL DEFAULT 'VND'	ISO 4217 code; VND for this deployment.
Status	ENUM NOT NULL	<i>active, frozen, closed.</i>
OpenedAt	DATE NOT NULL DEFAULT CURRENT_DATE	Opening date.

8.6 Table Transactions

Transactions form the append-only ledger. The **Amount** constraint enforces positivity — direction is encoded in **TxType**, not the sign of the amount, which simplifies reporting queries. **BalanceAfter** captures a snapshot of the source account’s balance immediately after the transaction is committed, providing a self-consistent audit trail without requiring a full replay of history.

Table 9: Transactions

Column	Type	Notes
TransactionID	BIGINT PK AUTO_INCREMENT	BIGINT to accommodate high-volume append.
AccountID	INT FK → Accounts NOT NULL	Primary leg (source for transfers).
RelatedAccount	INT FK → Accounts	Counter-party account for transfers; NULL for deposits/withdrawals.
TxType	ENUM NOT NULL	<i>deposit, withdrawal, transfer, fee, interest.</i>
Amount	DECIMAL(18,2) NOT NULL CHECK > 0	Always positive; direction is inferred from type.
BalanceAfter	DECIMAL(18,2) NOT NULL	Source account balance after commit.
Description	VARCHAR(255)	Optional memo field.
PerformedBy	INT FK → Users NOT NULL	Acting user for audit attribution.
Status	ENUM NOT NULL	<i>pending, completed, failed, reversed.</i>
CreatedAt	TIMESTAMP DEFAULT NOW()	Server timestamp; not client-supplied.

8.7 Table Loans

The **MonthlyPayment** value is computed at issuance using the standard annuity formula and stored rather than recalculated on every query, which avoids floating-point drift over a long repayment horizon. **OutstandingBal** is decremented on each payment and drives the *paid* status transition.

Table 10: Loans

Column	Type	Notes
LoanID	INT PK AUTO_INCREMENT	Surrogate key.
CustomerID	INT FK → Customers NOT NULL	Borrower.
Principal	DECIMAL(18,2) NOT NULL	Disbursed amount.
InterestRate	DECIMAL(5,2) NOT NULL	Annual percentage rate.
TermMonths	INT NOT NULL	Repayment horizon in months.
MonthlyPayment	DECIMAL(18,2) NOT NULL	Pre-computed annuity instalment.
OutstandingBal	DECIMAL(18,2) NOT NULL	Decrement on each repayment.
Status	ENUM NOT NULL	<i>pending, active, paid, defaulted.</i>
StartDate	DATE NOT NULL	Disbursement date.
EndDate	DATE NOT NULL	Scheduled end of repayment.

8.8 Table CreditCards and CardTransactions

Credit cards are modelled as a separate product from accounts. The **CreditLimit** sets the maximum exposure the bank accepts, and **OutstandingBal** tracks the current debt. Card transactions are stored in **CardTransactions** rather than the main **Transactions** table because they have a different shape (merchant name, no **BalanceAfter** snapshot) and a different reporting lifecycle.

Table 11: CreditCards

Column	Type	Notes
CardID	INT PK AUTO_INCREMENT	Surrogate key.
CardNumber	VARCHAR(20) UNIQUE NOT NULL	Masked to last four digits on display.
CustomerID	INT FK → Customers NOT NULL	Cardholder.
CreditLimit	DECIMAL(18,2) NOT NULL	Maximum outstanding balance allowed.
OutstandingBal	DECIMAL(18,2) NOT NULL DE- FAULT 0	Current debt; checked against limit on charge.
ExpiryDate	DATE NOT NULL	Card expiry; checked at transaction time.
Status	ENUM NOT NULL	<i>active, blocked, expired.</i>

CardTransactions stores every charge against a card with **CardID**, **Amount**, **Merchant**, **Description**, **Status**, and **CreatedAt**. The merchant field distinguishes card spending patterns, which is useful for fraud detection and customer spending reports.

8.9 Table AuditLog

The **AuditLog** is the most important table from a compliance perspective. It is written exclusively by database triggers, not by application code, which means no service layer bug or missing **try** block can cause an operation to go unlogged. The **OldData** and **NewData** JSON columns capture the full before/after state so that auditors can reconstruct exactly what changed and when, without running queries against production tables.

Table 12: AuditLog

Column	Type	Notes
AuditID	BIGINT PK AUTO_INCREMENT	Append-only; never updated or deleted.
TableName	VARCHAR(64) NOT NULL	Identifies which table was mutated.
RecordID	VARCHAR(64) NOT NULL	The primary key of the affected row.
Action	ENUM NOT NULL	<i>INSERT, UPDATE, DELETE.</i>
OldData	JSON	Full previous state (NULL for INSERT).
NewData	JSON	Full resulting state (NULL for DELETE).
PerformedBy	INT FK → Users	Read from session variable @app_user_id.
IpAddress	VARCHAR(45)	Captured from the HTTP request when available.
CreatedAt	TIMESTAMP DE- FAULT NOW()	Server-side event timestamp.

9 Transactional Integrity and Automation

9.1 Stored Procedure: Atomic Fund Transfer

The `sp_transfer` procedure is the canonical and only correct way to move money between two accounts. It acquires an exclusive row lock on the source account before reading the balance, which prevents a race condition where two concurrent transfers could both read the same pre-deduction balance and together overdraw the account. If the balance is insufficient, a `SIGNAL` raises an application-level exception that the `EXIT HANDLER` catches, rolls back the transaction, and re-raises to the caller.

```

1 DELIMITER $$
2 CREATE PROCEDURE sp_transfer(
3     IN p_from INT, IN p_to INT,
4     IN p_amount DECIMAL(18,2), IN p_user INT)
5 BEGIN
6     DECLARE v_bal DECIMAL(18,2);
7     DECLARE EXIT HANDLER FOR SQLEXCEPTION
8         BEGIN ROLLBACK; RESIGNAL; END;
9 
```

```
10 START TRANSACTION;
11 -- Lock source row to serialise concurrent transfers
12 SELECT Balance INTO v_bal
13 FROM Accounts WHERE AccountID = p_from FOR UPDATE;
14
15 IF v_bal < p_amount THEN
16     SIGNAL SQLSTATE '45000'
17     SET MESSAGE_TEXT = 'Insufficient funds for transfer';
18 END IF;
19
20 UPDATE Accounts SET Balance = Balance - p_amount WHERE AccountID =
    p_from;
21 UPDATE Accounts SET Balance = Balance + p_amount WHERE AccountID =
    p_to;
22
23 INSERT INTO Transactions(AccountID, RelatedAccount, TxType,
24     Amount, BalanceAfter, PerformedBy, Status)
25 VALUES (p_from, p_to, 'transfer', p_amount,
26     v_bal - p_amount, p_user, 'completed');
27 COMMIT;
28 END$$
29 DELIMITER ;
```

Listing 3: sp_transfer – atomic inter-account transfer

9.2 Trigger: Automated Audit Logging

Every AFTER UPDATE trigger on a ledger table appends a row to AuditLog with the before and after state serialised as JSON. The user attribution comes from the MySQL session variable @app_user_id, which the Flask request hook sets after JWT validation. This design decouples the audit system from the application: even a direct SQL client connecting to MySQL in the development environment will populate the audit log, as long as it sets the session variable.

```
1 CREATE TRIGGER trg_audit_accounts_upd
2 AFTER UPDATE ON Accounts FOR EACH ROW
3 INSERT INTO AuditLog(TableName, RecordID, Action,
4    OldData, NewData, PerformedBy)
5 VALUES ('Accounts', OLD.AccountID, 'UPDATE',
6     JSON_OBJECT('Balance', OLD.Balance, 'Status', OLD.Status),
7     JSON_OBJECT('Balance', NEW.Balance, 'Status', NEW.Status),
8     @app_user_id);
```

Listing 4: Audit trigger on Accounts

9.3 Indexes and Views

Indexes are placed on every foreign key column, on `Accounts.AccountNumber`, `Customers.IDNumber`, `Users.Username`, and the composite `Transactions(AccountID, CreatedAt)` to accelerate the most common query patterns: recent transactions for a given account, login lookup, and KYC search by ID number. Without the composite index on `Transactions`, fetching the last hundred transactions for a high-volume account would require a full table scan.

Reporting endpoints query a small set of denormalised views — `v_branch_performance`, `v_top_customers`, and others — to keep the SQL in the route handlers thin and to provide a stable interface that can be optimised independently of the application code.

10 Backend API

10.1 Project Layout

The backend follows a strict three-layer pattern: routes receive HTTP requests and return HTTP responses; services contain all business logic and speak only Python types; the database layer executes SQL. This separation means that the CLI can call service functions directly, and that services can be unit-tested without spinning up a Flask application.

```

1 backend/
2   app.py                # create_app() factory + error handlers
3   config.py             # Env-driven settings
4   api_response.py       # ok / fail / paginate helpers
5   routes/               # 11 blueprints, one per resource
6     auth_routes.py      customer_routes.py    account_routes.py
7     transaction_routes.py employee_routes.py  branch_routes.py
8     loan_routes.py      card_routes.py        report_routes.py
9     audit_routes.py     ml_routes.py
10  services/              # Pure business logic, called by routes
11    auth_service.py      transaction_service.py
12    fraud_service.py     report_service.py
13  ml/                    # Model training + prediction
14    fraud_detection_model.py
15  utils/
16    query_builder.py     # Whitelisted dynamic filters / sort
17  core/
18    auth.py              # JWT issue / decode / @auth_required
19    db_connection.py     # Connection pool + context-managed cursor
20    validators.py        formatters.py

```

Listing 5: Backend layout

10.2 REST Endpoint Catalogue

Eleven blueprints are mounted under `/api`. Only `/api/health` and `/api/auth/login` are public; every other endpoint requires a valid Bearer token, validated by the `@auth_required` decorator before the route handler is invoked.

Table 13: REST endpoint catalogue (selection)

Endpoint	Method	Purpose
<code>/api/health</code>	GET	Liveness probe used by <code>start.sh</code> .
<code>/api/auth/login</code>	POST	Returns JWT + user payload.
<code>/api/auth/me</code>	GET	Current token claims.
<code>/api/auth/logout</code>	POST	Stateless — client discards token.
<code>/api/customers</code>	GET/POST	List (paginated, filtered) / create customers.
<code>/api/customers/<id></code>	GET/PUT/DELETE	Customer detail.
<code>/api/accounts</code>	GET/POST	List / open accounts.
<code>/api/accounts/<id></code>	GET/PUT/DELETE	Account detail and status change.
<code>/api/transactions</code>	GET	Paginated, filtered, sortable transaction history.
<code>/api/transactions/deposit</code>	POST	Atomic deposit.
<code>/api/transactions/withdraw</code>	POST	Atomic withdrawal.
<code>/api/transactions/transfer</code>	POST	Atomic inter-account transfer via <code>sp_transfer</code> .
<code>/api/transactions/fraud/<i</code>	GET	Rule-based fraud flags for an account.
<code>/api/loans</code>	GET/POST	Loan applications and schedules.
<code>/api/cards</code>	GET/POST	Credit cards and card transactions.
<code>/api/employees</code>	GET/POST/PUT/DE	Staff management.
<code>/api/branches</code>	GET/POST/PUT/DE	Branch management.
<code>/api/reports/top</code>	GET	Top customers by balance.
<code>/api/reports/branch</code>	GET	Branch performance metrics.
<code>/api/audit</code>	GET	Paginated audit log with action-type filters.
<code>/api/ml/predict-fraud</code>	POST	Fraud probability (rule fallback if model absent)
<code>/api/ml/status</code>	GET	Model readiness check.

10.3 Application Factory

Using a factory function (`create_app`) rather than a module-level Flask instance means the application can be instantiated multiple times — once for the production server, once for tests with a different configuration — without global state bleeding between instances. Error handlers are registered on the factory so that every unhandled exception returns a structured JSON response rather than an HTML traceback.

```
1 def create_app() -> Flask:
2     app = Flask(__name__)
3     app.config.from_object(Config)
4     CORS(app, resources={r"/api/*": {"origins": "*"}},
5         supports_credentials=True)
6     Limiter(app=app, key_func=get_remote_address,
7             default_limits=["200 per minute"])
8
9     app.register_blueprint(auth_bp, url_prefix="/api/auth")
10    app.register_blueprint(customer_bp, url_prefix="/api/customers")
11    app.register_blueprint(account_bp, url_prefix="/api/accounts")
12    app.register_blueprint(tx_bp, url_prefix="/api/transactions")
13
14    # ... 7 more blueprints
15
16    @app.get("/api/health")
17    def health():
18        return {"success": True, "data": {"status": "ok"}, "message":
19            "alive"}
20
21    @app.errorhandler(Exception)
22    def _err(e):
23        traceback.print_exc()
24        return fail(str(e) or "Internal error", 500)
25
26    return app
```

Listing 6: backend/app.py – factory and global error handlers

10.4 Authentication Flow

The login sequence is intentionally simple and stateless. The browser sends credentials once, receives a signed JWT, and presents that token on every subsequent request. The server validates the signature and expiry on every request without consulting a session store. This makes the API horizontally scalable: any Flask instance can validate any token, because they all share the same `JWT_SECRET`.



Figure 4: JWT authentication sequence

10.5 Login Handler

The login route validates the presence of credentials, fetches the matching active user record, checks the password hash, updates `LastLoginAt`, and issues the token. Returning an identical error message for both “user not found” and “wrong password” is deliberate: it prevents username enumeration by an attacker timing the responses.

```

1 @bp.post("/login")
2 def login():
3     data = request.get_json(silent=True) or {}
4     username, password = data.get("username"), data.get("password")
5     if not username or not password:
6         return fail("username & password required", 400)
7
8     with get_cursor() as cur:
9         cur.execute("SELECT * FROM Users WHERE Username=%s AND
10                    IsActive=1",
11                    (username,))
12         user = cur.fetchone()
13
14     if not user or not check_password_hash(user["PasswordHash"],
15     password):
16         return fail("Invalid credentials", 401)
17
18     with get_cursor(commit=True) as cur:
19         cur.execute("UPDATE Users SET LastLoginAt=NOW() WHERE UserID=%s",
20                    (user["UserID"],))
21
22     token = issue_token(user)
23     return ok({"token": token,
24              "user": {"id": user["UserID"],
25                      "username": user["Username"],
26                      "role": user["Role"]}}, "Login successful")
  
```

Listing 7: backend/routes/auth_routes.py

10.6 Token Issuing and the auth_required Decorator

The `auth_required` decorator validates the JWT signature and expiry on every protected request, parks the decoded claims in Flask’s `g` context for downstream use, and

optionally gates access to specific roles. The `roles` parameter is available across all route handlers; for the demonstration build, the seeded users do not exercise the role boundary because all three sample accounts are equally acceptable to all screens in the SPA.

```

1 def issue_token(user: dict) -> str:
2     payload = {
3         "uid": user["UserID"],
4         "uname": user["Username"],
5         "role": user["Role"],
6         "exp": dt.datetime.utcnow() + dt.timedelta(hours=
JWT_EXPIRE_HOURS),
7         "iat": dt.datetime.utcnow(),
8     }
9     return jwt.encode(payload, JWT_SECRET, algorithm="HS256")
10
11 def auth_required(roles: list[str] | None = None):
12     def deco(fn):
13         @wraps(fn)
14         def wrapper(*a, **kw):
15             auth = request.headers.get("Authorization", "")
16             if not auth.startswith("Bearer "):
17                 return jsonify({"success": False,
18                                 "message": "Missing token"}), 401
19             try:
20                 payload = jwt.decode(auth.split(" ", 1)[1],
JWT_SECRET, algorithms=[JWT_ALG])
21             except jwt.ExpiredSignatureError:
22                 return jsonify({"success": False,
23                                 "message": "Token expired"}), 401
24             except jwt.InvalidTokenError:
25                 return jsonify({"success": False,
26                                 "message": "Invalid token"}), 401
27             if roles and payload["role"] not in roles:
28                 return jsonify({"success": False,
29                                 "message": "Forbidden"}), 403
30             g.current_user = payload
31             return fn(*a, **kw)
32         return wrapper
33     return deco
34

```

Listing 8: core/auth.py

11 Services Layer

The `services/` package contains all pure business logic — no HTTP types, no JSON serialisation. This means it can be unit-tested in isolation, called from the Flask routes,

and equally called from the Python CLI without any framework dependency. The four most significant services are described below.

11.1 transaction_service

The transaction service wraps `sp_transfer` for inter-account transfers, and provides deposit and withdraw routines that acquire an exclusive row lock on the target account, validate the balance, and insert the resulting transaction row within a single `get_cursor(commit=True)` context. By centralising balance mutation here, the code-base has a single place to add new validation rules — such as daily transfer limits or frozen account checks — without hunting across route handlers.

11.2 fraud_service

The fraud detection service performs static historical analysis on previously recorded transaction data. Because the system operates on a seeded dataset rather than a real-time event stream, fraud identification is based on retrospective behavioural patterns, making it suitable for internal audit and account review workflows. Detection logic is implemented directly against the `Transactions` table through a set of deterministic rules. These include burst activity detection, where five or more transactions occur within a one-hour period; amount spike analysis, where the current day's maximum transaction exceeds ten times the rolling 30-day average; and late-night withdrawal monitoring, which flags accounts performing three or more withdrawals between midnight and 5 a.m. within seven days. The service can optionally integrate a `RandomForest` classifier defined in `fraud_detection_model.py` when trained model artefacts are available. If the model cannot be loaded, the system gracefully falls back to a lightweight ratio-based heuristic so fraud analysis remains operational without external ML dependencies.

11.3 report_service

The report service issues SQL aggregations against the schema's reporting views and returns Python dictionaries shaped for direct binding by the SPA's chart components. Keeping this logic in a service rather than inline in a route handler means the CLI can produce the same reports in terminal-formatted tables using the `rich` library.

12 Frontend

12.1 Project Layout

The frontend is a TypeScript React application. The directory structure mirrors the backend's layered approach: `services/` wraps every API call, `contexts/` manages shared state, `hooks/` encapsulates reusable logic, and `routes/` contains one file per page.

```

1 frontend/src/
2   main.tsx
3   routeTree.tsx
4   index.css                                # Tailwind directives + theme
5   contexts/AuthContext.tsx                # JWT in memory + localStorage
6   services/
7     api.ts                                # axios instance + interceptors
8     authService.ts                         # login / logout / me
9   components/
10    AppLayout.tsx                          # Sidebar + topbar + Outlet
11    Protected.tsx                          # Auth + role guard
12    Modal.tsx StatCard.tsx Pagination.tsx FilterBar.tsx Skeleton.tsx
13    charts/
14      MoneyFlowSankey.tsx                  # D3 Sankey
15      NetworkGraph.tsx                    # D3 force-directed
16      TransactionHeatmap.tsx               # Custom heatmap
17    hooks/
18      useDebounce.ts useFilters.ts
19    lib/
20      utils.ts validations.ts              # cn() + Zod schemas
21    routes/
22      login.tsx
23      dashboard.tsx
24      customers.tsx accounts.tsx transactions.tsx
25      employees.tsx branches.tsx loans.tsx cards.tsx
26      audit.tsx
27      reports.tsx reports.advanced.tsx

```

Listing 9: Frontend layout

12.2 Routing and Layout

The application uses TanStack React Router declaratively with no server-side rendering. `AppLayout` renders the sidebar, the topbar with the current user's name and role, and a TanStack `<Outlet/>` that the router replaces with the active route component. `Protected` wraps every authenticated route: it reads the JWT from `AuthContext` and redirects to `/login` when the token is absent or expired, so no protected route component ever renders for an unauthenticated user.

12.3 Auth Context and HTTP Client

A React context exposes the current user object and the JWT to any component in the tree. The context persists both values to `localStorage` on login and clears them on logout. An axios instance reads the token on every request via an interceptor, so route components and service modules never manually handle the `Authorization` header. A response interceptor catches HTTP 401 responses, clears the stored token, and redirects to `/login`, centralising the session-expiry behaviour.

```
1 export const api = axios.create({ baseUrl: import.meta.env.  
    VITE_API_URL });  
2  
3 api.interceptors.request.use((config) => {  
4     const token = localStorage.getItem("token");  
5     if (token) config.headers.Authorization = `Bearer ${token}`;  
6     return config;  
7 });  
8  
9 api.interceptors.response.use(  
10     (r) => r,  
11     (err) => {  
12         if (err.response?.status === 401) {  
13             localStorage.removeItem("token");  
14             window.location.href = "/login";  
15         }  
16         return Promise.reject(err);  
17     }  
18 );
```

Listing 10: `services/api.ts` – shared axios client

12.4 Forms and Validation

Every create and edit form uses React Hook Form bound to a Zod schema defined in `lib/validations.ts`. A single resolver provides both runtime field-level validation (with error messages rendered next to each input) and TypeScript types for the form values, so the compiler catches schema mismatches before the code runs. For example, the transfer form's Zod schema enforces that `amount` is a positive number and that `fromAccountId` and `toAccountId` are distinct, before the request reaches the API.

12.5 Server-State Management

TanStack Query owns all data fetched from the API. Lists use `useQuery` with paginated query keys that encode the current page, filter values, and sort column, so navigating

between pages or changing a filter automatically triggers a new fetch without any manual state management. Mutations (`useMutation`) invalidate the relevant query keys on success, so the list view immediately reflects the newly created or updated record.

12.6 Visualisations

Standard charts — line, bar, and donut — are built with Recharts, which provides React-idiomatic bindings and responsive containers without requiring D3 knowledge. Three advanced visualisations use D3 directly. The **MoneyFlowSankey** chart aggregates inter-branch transfer volumes into a Sankey diagram, making it immediately visible which branches are net senders and which are net receivers of funds. The **NetworkGraph** renders a force-directed graph of customer-to-account-to-counter-party links, useful for identifying clusters of accounts that frequently transact with each other. The **TransactionHeatmap** is a GitHub-contribution-style calendar heatmap of daily transaction volume, surfacing seasonal patterns and anomalous quiet periods.

12.7 Visual Snapshots

The following screenshots cover every major screen in the React SPA, demonstrating how the frontend consumes the REST API and presents the underlying database entities to bank staff.

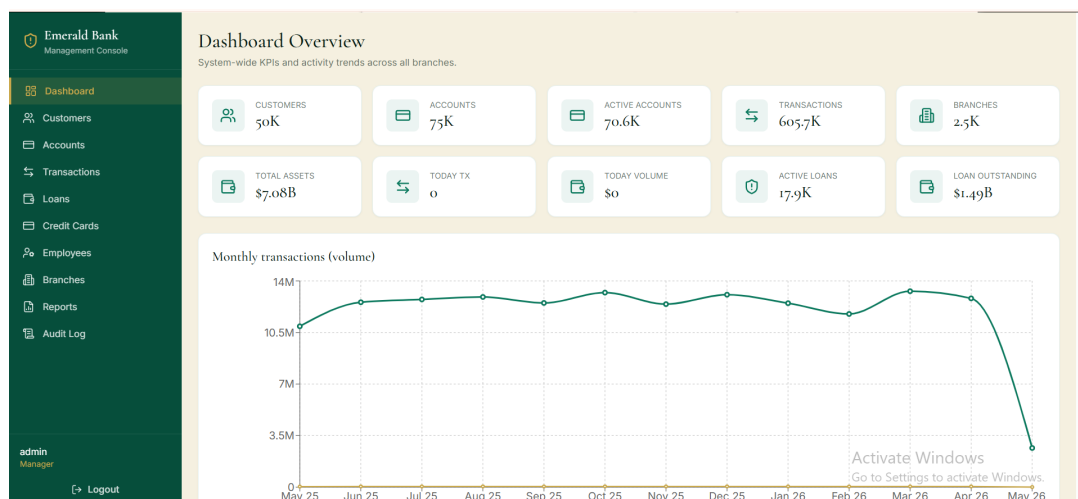


Figure 5: Operational dashboard showing KPI stat cards (total customers, accounts, assets, and daily transactions) sourced from the reporting views via `/api/reports/top`.

Emerald Bank
Management Console

Dashboard

Customers

Accounts

Transactions

Loans

Credit Cards

Employees

Branches

Reports

Audit Log

admin
Manager

Logout

Customers

Search by ID, name, email, phone, IDNumber...

Total Balance > 1 Accounts > City contains Name contains

Sort by: default Clear all

ID	Name	Email	Phone	City	Accounts	Total Balance	Actions
38838	Makyla Curtis	cust38818.c986fb@email.com	028-197-2751	Jennamouth	1	\$19,009	View Edit Delete
38837	Brandon Schroeder	tpowell@example.org	688-861-2744	East Joshua	1	\$119,506	View Edit Delete
38836	James Stein	cust38816.d9d559@email.com	064-280-3796	Port Kristamouth	1	\$5,790	View Edit Delete
38835	Jeremy Forbes	christianwhite@example.net	009-941-8946	North Taraborough	1	\$148,488	View Edit Delete
38834	Jacob Gonzalez	ramosmofly@example.org	446-161-3236	North Jeremybury	1	\$154,145	View Edit Delete
38833	Kevin Smith	cust38813.7f842c@email.com	854-967-2287	East Greg	1	\$125,666	View Edit Delete
38831	Brett Werner	grodriquez@example.org	733-163-7040	South Ana	1	\$141,105	View Edit Delete
38830	David Nash	cust38810.d8fbf0@email.com	372-452-3329	Timothychester	1	\$89,031	View Edit Delete
38827	Jose Davis	christianacruz@example.com	879-592-6209	South Connieberg	1	\$44,440	View Edit Delete
38826	Nicholas Wheeler	ann32@example.com	923-399-4007	Murraybury	1	\$67,072	View Edit Delete
38825	William Parker	shawmacollins@example.net	161-612-7253	Perezbury	1	\$115,623	View Edit Delete
38824	Lisa Hester	matthewkerr@example.net	810-390-9148	Latahafort	1	\$45,796	View Edit Delete
38823	Kevin Mendoza	frank76@example.org	611-835-8708	Simmonsfort	1	\$163,169	View Edit Delete
38822	Matthew Gutierrez	chambersandrew@example.com	601-757-9129	West Vickitown	1	\$12,344	View Edit Delete
38821	Jessica Price	cust38801.9a7d89@email.com	677-017-3026	Manningmouth	1	\$179,840	View Edit Delete
38820	Todd Todd	joshua316@example.com	732-252-4173	Christopherborough	1	\$165,684	View Edit Delete

Figure 6: Customer list with server-side pagination, debounced full-text search, and city/balance filters delegated to `utils/query_builder.py` on the backend.

Emerald Bank
Management Console

Dashboard

Customers

Accounts

Transactions

Loans

Credit Cards

Employees

Branches

Reports

Audit Log

admin
Manager

Logout

Accounts

Search by account ID, number or customer name...

Balance > Type >

Sort by: default Clear all

ID	Number	Customer	Branch	Type	Balance	Status	Actions
75020	ACC020075000	David Duarte	Herreraestad Branch	savings	\$63,893	active	Detail Freeze Close
75019	ACC020074999	James Lopez	Halemouth Branch	savings	\$87,441	active	Detail Freeze Close
75018	ACC020074998	Makyla Curtis	Chapmanport Branch	savings	\$19,009	active	Detail Freeze Close
75017	ACC020074997	Olivia May	Angelaort Branch	fixed	\$13,904	active	Detail Freeze Close
75016	ACC020074996	Jonathon Smith	Wilsonmouth Branch	savings	\$64,096	active	Detail Freeze Close
75015	ACC020074995	Brandon Schroeder	Townsendburgh Branch	checking	\$119,506	active	Detail Freeze Close
75014	ACC020074994	David Jimenez	Keithchester Branch	checking	\$3,880	active	Detail Freeze Close
75013	ACC020074993	Colin Lopez	Marvinton Branch	checking	\$192,681	active	Detail Freeze Close
75012	ACC020074992	Michael Jacobs	Lake Bradley Branch	checking	\$199,538	active	Detail Freeze Close
75011	ACC020074991	Manuel Nguyen	East Susanville Branch	savings	\$0	closed	Detail
75010	ACC020074990	Anita Holland	New Kimberlystad Branch	savings	\$150,119	active	Detail Freeze Close
75009	ACC020074989	Frank Johnson	Port Melissaad Branch	savings	\$16,603	active	Detail Freeze Close
75008	ACC020074988	Shawn Elliott	Connorville Branch	checking	\$70,358	active	Detail Freeze Close
75007	ACC020074987	James Stein	North Mark Branch	savings	\$5,790	active	Detail Freeze Close
75006	ACC020074986	Christopher Wilson	New Ruth Branch	savings	\$79,178	active	Detail Freeze Close

Figure 7: Accounts page with inline Freeze and Close actions. Each status change calls `PUT /api/accounts/<id>` and is captured by the `AuditLog` trigger.

ID	Account	Customer	Type	Amount	Balance After	When
540148	ACC020023211	Cynthia Fletcher	deposit	\$21	\$0	06/05/2026, 20:59:44
471623	ACC020006435	Jennifer Stephens	withdrawal	\$348	\$0	06/05/2026, 20:59:10
416450	ACC020033920	Cesar Bennett	withdrawal	\$54	\$0	06/05/2026, 20:57:53
46669	ACC020013696	Katelyn Grant	transfer	\$95	\$0	06/05/2026, 20:56:07
186071	ACC020002869	Jason Jackson	withdrawal	\$5	\$0	06/05/2026, 20:55:32
263570	ACC020064798	Tara Snow	withdrawal	\$54	\$0	06/05/2026, 20:54:27
308883	ACC020014262	Christopher Petersen	transfer	\$17	\$0	06/05/2026, 20:54:18
451869	ACC020047164	Chad Miller	fee	\$5	\$0	06/05/2026, 20:53:02
74929	ACC020020285	Amanda Tucker	transfer	\$19	\$0	06/05/2026, 20:50:02
41505	ACC020005622	Harry Dennis	transfer	\$343	\$0	06/05/2026, 20:50:02
63292	ACC020031910	Lisa Donaldson	transfer	\$202	\$0	06/05/2026, 20:48:41
217848	ACC0200009176	Patrick Garcia	withdrawal	\$193	\$0	06/05/2026, 20:46:51
437055	ACC0200048457	Richard Moore	withdrawal	\$46	\$0	06/05/2026, 20:46:51

Figure 8: Transaction ledger displaying the append-only Transactions table with type badges, counter-party account, amount, and BalanceAfter snapshot for audit traceability.

ID	Customer	Principal	Rate %	Term	Outstanding	Status	Actions
30003	Thomas Howard	\$2,263	9.19	36m	\$2,263	active	Interest Pay
30002	Kathleen Haas	\$96,599	11.4	36m	\$96,599	active	Interest Pay
30001	Christopher Meyer	\$65,764	6.24	120m	\$65,764	closed	Interest
30000	Zachary Sandoval	\$70,458	12.27	48m	\$70,458	pending	Interest Pay
29999	Cheryl Maxwell	\$40,343	14.21	120m	\$40,343	active	Interest Pay
29998	Michelle Pearson	\$68,744	8.76	24m	\$68,744	closed	Interest
29997	James Martinez	\$102,534	12.84	48m	\$102,534	pending	Interest Pay
29996	Joseph Moore	\$128,415	14.88	60m	\$128,415	closed	Interest
29995	Ethan Gordon	\$289,249	4.34	120m	\$289,249	pending	Interest Pay
29994	Elaine Williams	\$84,367	7.55	12m	\$84,367	active	Interest Pay
29993	Keith Johnson	\$263,122	7.74	36m	\$263,122	active	Interest Pay
29992	Gary Barnes	\$275,507	12.04	60m	\$275,507	closed	Interest
29991	Luke Green	\$187,418	12.04	60m	\$187,418	active	Interest Pay
29990	Brian Thomas	\$254,623	4.08	120m	\$254,623	active	Interest Pay
29989	Mike Waters	\$278,244	6.51	36m	\$278,244	active	Interest Pay

Figure 9: Loans page listing principal, interest rate, term, outstanding balance, and status. The monthly payment column shows the pre-computed annuity stored at issuance in Loans.MonthlyPayment.

Emerald Bank
Management Console

Dashboard

Customers

Accounts

Transactions

Loans

Credit Cards

Employees

Branches

Reports

Audit Log

admin
Manager

Logout

Credit Cards

Search by card ID, number or customer name...

Limit: Outstanding: Status: Has tx:

Sort by:

default

 Clear all

ID	Number	Customer	Limit	Outstanding	Expiry	Status	Actions
100003	1784216474722391	Corey Moore	\$4,314	\$2,660	23/04/2025, 07:00:00	active	History Block
100002	1573852597096693	Brandon Mejia	\$21,784	\$9,209	27/11/2029, 07:00:00	active	History Block
100001	9868721961684967	Alexander Gilmore	\$42,929	\$26,540	18/02/2025, 07:00:00	active	History Block
100000	247742127232831	Patrick Johnson	\$23,134	\$4,640	31/10/2031, 07:00:00	active	History Block
99999	6888410025472234	Raymond Lynch	\$24,313	\$3,038	05/01/2029, 07:00:00	active	History Block
99998	1262589973049186	Jordan Wilson	\$20,825	\$4,525	05/10/2032, 07:00:00	active	History Block
99997	8357538201842187	Anthony Dean	\$37,163	\$24,494	03/11/2028, 07:00:00	active	History Block
99996	5193426332720562	Kristina Bowers	\$16,686	\$5,768	24/12/2032, 07:00:00	active	History Block
99995	1630595565141198	Stephen Wilson	\$23,224	\$12,454	08/07/2029, 07:00:00	active	History Block
99994	7398446632828646	Jennifer Wilson	\$27,799	\$614	10/02/2030, 07:00:00	active	History Block
99993	0803156519033269	Erica Brown	\$22,342	\$17,737	14/01/2032, 07:00:00	active	History Block
99992	563495218246539	Rhonda Davis	\$13,603	\$7,727	26/02/2030, 07:00:00	active	History Block
99991	1462010714838273	Luis Taylor	\$45,810	\$31,007	03/12/2026, 07:00:00	active	History Block
99990	6653993892326588	Meghan Giles	\$42,510	\$23,337	27/05/2032, 07:00:00	active	History Block
99989	9004155384008424	Michael Austin	\$11,853	\$4,876	27/06/2028, 07:00:00	active	History Block

Figure 10: Credit Cards page with limit, outstanding balance, expiry, and status per card. The Block action triggers an AuditLog entry via the CreditCards update trigger.

Emerald Bank
Management Console

Dashboard

Customers

Accounts

Transactions

Loans

Credit Cards

Employees

Branches

Reports

Audit Log

admin
Manager

Logout

Employees

Search by ID, name, email, position, branch...

Salary: Status:

Sort by:

default

 Clear all

ID	Name	Email	Position	Branch	Salary	Status	Actions
5009	Elizabeth Patterson	elizabeth.patterson@bank.local	Cashier	Rollinsburgh Branch	\$1,259	active	Edit Delete
5008	Mark Sullivan	mark.sullivan@bank.local	Cashier	North Carrie Branch	\$1,150	active	Edit Delete
5007	Thomas Herrera	thomas.herrera@bank.local	Teller	Port Gloria Branch	\$1,587	active	Edit Delete
5006	Madison Garcia	madison.garcia@bank.local	Cashier	Smithfort Branch	\$1,013	active	Edit Delete
5005	Michael Caldwell	michael.caldwell@bank.local	Cashier	Smithborough Branch	\$1,033	active	Edit Delete
5004	Victoria Barron	victoria.barron@bank.local	Auditor	Smithside Branch	\$2,781	active	Edit Delete
5003	Andrea Hill	andrea.hill@bank.local	Loan Officer	Salgon Central	\$2,718	active	Edit Delete
5002	Brian Cooper	brian.cooper2@bank.local	Teller	Lake Janetstad Branch	\$1,470	active	Edit Delete
5001	Rebecca Trevino	rebecca.trevino@bank.local	Teller	West Jiliantown Branch	\$1,576	active	Edit Delete
5000	Michael Anderson	michael.anderson2@bank.local	Manager	Hendersonsberg Branch	\$4,507	active	Edit Delete
4999	Sandy English	sandy.english@bank.local	Auditor	Reynoldstown Branch	\$2,757	active	Edit Delete
4998	Crystal Moody	crystal.moody@bank.local	Auditor	East Gregoryside Branch	\$3,124	active	Edit Delete
4997	Danielle Becker	danielle.becker@bank.local	Teller	Perezborough Branch	\$1,330	active	Edit Delete
4996	Matthew Kirk	matthew.kirk@bank.local	Loan Officer	West Emily Branch	\$1,900	active	Edit Delete
4995	Shannon Pittman	shannon.pittman@bank.local	Cashier	Davidsonview Branch	\$984	active	Edit Delete

Figure 11: Employee management filtered by salary and status. Terminated staff remain visible for audit traceability; their linked Users record is soft-disabled.

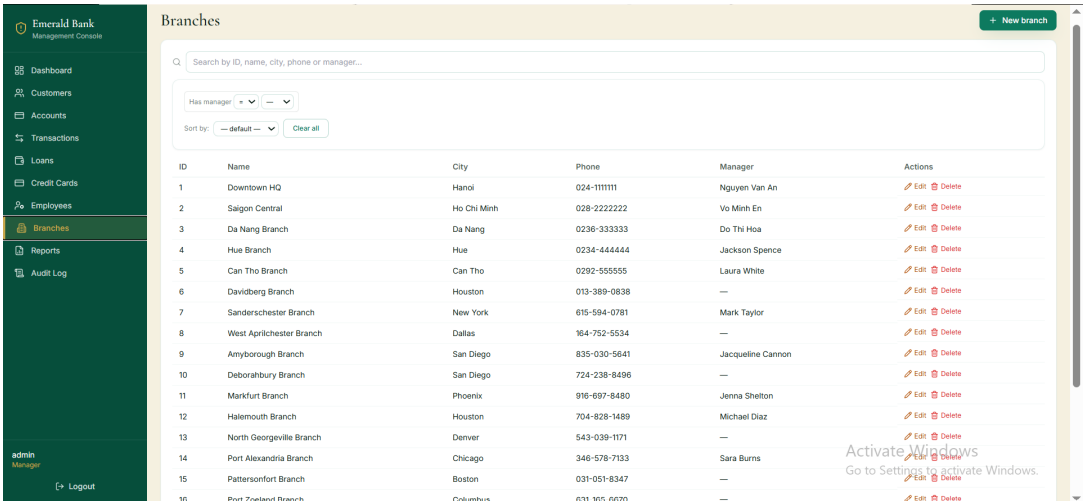


Figure 12: Branches page listing every office with city, phone, and manager. The nullable ManagerID uses ON DELETE SET NULL to survive staff departures.

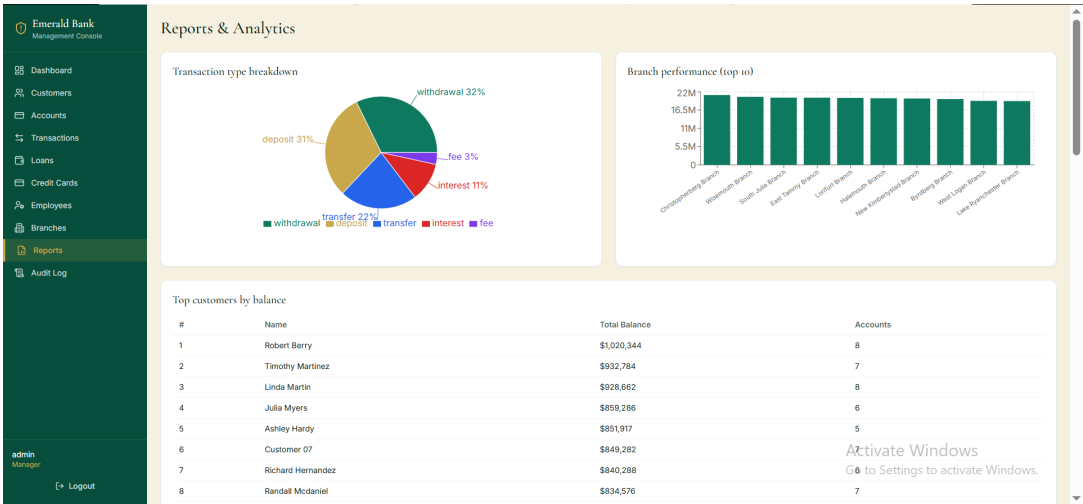


Figure 13: Reports and Analytics page with transaction-type donut, top-10 branch bar chart, and top customers by balance table, all served by /api/reports/ endpoints.

ID	Table	Record	Action	User	When
607889	Accounts	75011	UPDATE		06/05/2026, 14:01:16
607888	Accounts	74993	UPDATE		06/05/2026, 14:01:16
607887	Accounts	74945	UPDATE		06/05/2026, 14:01:16
607886	Accounts	74884	UPDATE		06/05/2026, 14:01:16
607885	Accounts	74871	UPDATE		06/05/2026, 14:01:16
607884	Accounts	74821	UPDATE		06/05/2026, 14:01:16
607883	Accounts	74815	UPDATE		06/05/2026, 14:01:16
607882	Accounts	74805	UPDATE		06/05/2026, 14:01:16
607881	Accounts	74767	UPDATE		06/05/2026, 14:01:16
607880	Accounts	74713	UPDATE		06/05/2026, 14:01:16
607879	Accounts	74664	UPDATE		06/05/2026, 14:01:16
607878	Accounts	74517	UPDATE		06/05/2026, 14:01:16
607877	Accounts	74512	UPDATE		06/05/2026, 14:01:16
607876	Accounts	74454	UPDATE		06/05/2026, 14:01:16
607875	Accounts	74444	UPDATE		06/05/2026, 14:01:16

Figure 14: Audit Log viewer exposing 607,889 entries (605,712 INSERTs, 2,177 UPDATEs) with filters by table, action type, and date. The diff modal renders OldData/NewData JSON side by side.

13 Security Posture

Security in this system is not an afterthought applied at the perimeter, but a property of each layer. The measures below work together to provide defence-in-depth appropriate for a course prototype, and each one is a direct precursor to a production control.

Password hashing uses `werkzeug.security.generate_password_hash` with the default `pbkdf2:sha256` algorithm and a per-user salt. Plaintext passwords are never stored and never appear in logs. Stateless JWT authentication with an 8-hour expiry means there are no server-side session stores to protect or synchronise. The JWT secret is read from `JWT_SECRET` in `.env` and never hardcoded.

Every SQL query uses bound parameters via the cursor's `%s` placeholders; dynamic filter and sort clauses pass through `utils/query_builder.py`, which whitelists the allowed column names so an attacker cannot inject an arbitrary expression into an `ORDER BY` clause. CORS is restricted to the SPA origin in production deployments. Flask-Limiter applies a 200 requests-per-minute default with tighter limits on `/auth/login` to slow credential-stuffing attacks. Mutating operations are recorded by triggers rather than application code, so no service can forget to log. MySQL's port 3306 is not published to the Docker host network, ensuring the database is unreachable from outside the container network.

14 Verification and Testing

14.1 Methodology

The project followed a hybrid waterfall-agile cycle. The schema, triggers, and stored procedures were designed upfront because changes to the data model cascade widely. The API, services, and frontend were built in iterative sprints with continuous integration of the components, allowing the frontend to be tested against the real API from the earliest stages.

Table 14: Development roadmap

Phase	Activity
1	Requirements & Analysis
2	Schema, Triggers, Stored Procedures
3	Flask API + Services
4	React SPA + Charts
5	Dockerise + One-click <code>start.sh</code>

14.2 Testing Strategy

Three layers of testing provide confidence at different levels of granularity. Unit tests in `backend/tests/` use `pytest` to exercise individual service functions and validation helpers with mocked database cursors, catching logic errors quickly without a running MySQL instance. Integration tests bring up the Flask API against a disposable MySQL container and exercise the full request cycle, including authentication, parameter validation, and database state. Concurrency tests simulate two clients simultaneously transferring funds out of the same account; the `FOR UPDATE` lock in `sp_transfer` serialises them, and the test asserts that the final balance reflects exactly one transfer, not both.

14.3 Manual Verification Scenarios

Four scenarios were verified manually during development to confirm that the system behaves correctly at its boundaries. Attempting to withdraw more than the available balance returns HTTP 400 with the message “Insufficient funds for transfer”. Calling any protected endpoint with an expired JWT returns HTTP 401 with “Token expired”; the SPA’s axios interceptor catches that response and redirects to `/login`. Sending more than 200 unauthenticated requests from a single IP within one minute returns

HTTP 429. After signing in as `admin` and updating an account's status, the resulting `AuditLog` row carries `PerformedBy = admin.UserID`, confirming that the session variable plumbing is working end-to-end.

15 Conclusion and Future Work

15.1 Summary

The report opened with the question of what it actually takes to move money safely inside a bank. The answer, laid out across the preceding sections, is a carefully coordinated stack of interlocking guarantees: a 3NF schema that prevents redundancy at rest, `FOR UPDATE` locks that prevent race conditions in flight, audit triggers that ensure no write goes unrecorded, and a stateless JWT layer that keeps every action attributed to an authenticated user.

The BMS delivers all of these as a complete, working system. The frontend wraps those operations in a clean SPA with typed forms, server-state caching, and three non-trivial D3 visualisations that would be at home in a real analytics dashboard. A Python CLI and a fraud-detection pipeline round out the deliverable. The entire stack boots from a single `./start.sh` command, and every design decision can be traced back to a requirement or a course concept — making the system not just functional, but explainable.

15.2 Future Roadmap

Several directions for future development are already visible from the current implementation. Differentiated UI behaviour per role is the most immediately valuable: the role claim is already in the JWT, so subsequent iterations can hide menus, gate routes, and tighten endpoint guards without changing the authentication layer. Refresh tokens would improve the session management story by replacing the current 8-hour access token with a short-lived access token paired with a long-lived refresh token stored in an `HttpOnly` cookie, eliminating the XSS exposure of `localStorage`. Production observability — structured logging, a Prometheus `/metrics` endpoint, and distributed request tracing — would be essential before the system handled real money. Field-level encryption of `IDNumber` and other PII at rest using InnoDB tablespace encryption would bring the system closer to GDPR compliance. Finally, the fraud model could be replaced with an isolation forest or gradient-boosted classifier trained on a larger labelled dataset, while retaining the current rule-based layer as a fast, explainable first pass.

Appendices

A Complete Project Directory Structure

```

1 banking-management-system/
2   core/                          db_connection.py, validators.py, formatters.py,
   auth.py
3   database/                      schema + views + procs + triggers + seed + audit
   + csv loader
4   backend/                      Flask API (routes/, services/, ml/, utils/,
   tests/)
5   frontend/                    React + Vite + TanStack Router (src/)
6   data_generation/             Kaggle downloader, transform scripts, csv_output
   /
7   python_cli/                 Rich-powered CLI (operations/, reports/,
   auth_cli.py)
8   docs/                       er_diagram, architecture_diagram, screenshots/,
   user_manual
9   tests/                      End-to-end TypeScript test specs
10  docker-compose.yml           mysql + backend + frontend
11  Dockerfile.backend           Dockerfile.frontend
12  .env.example                 .env.development .env.production
13  start.sh                    README.md

```

Listing 11: Top-level project tree

B Default Demonstration Credentials

Username	Password	Stored Role
admin	admin123	manager
teller1	teller123	teller
audit1	audit123	auditor